cs2040s — ssh donko@pe111.comp.nus.edu.sg — 255x72

CannotMoveException.java | Vehicle.java | Car.java | Foldable.java | Bike.java | RoadTrip.java

```java
interface Foldable {
  void fold();

  void unfold();
}
```

Foldable.java

```java
class CannotMoveException extends Exception {
  // public CannotMoveException() {
  // }
}
```

CannotMoveException.java

```java
abstract class Vehicle {
  public abstract void move(double dist) throws CannotMoveException;
}
```

Vehicle.java

"CannotMoveException.java" 4L, 91C written

```java
class Car extends Vehicle {
  public static final double FUEL_EFFICIENCY = 5.0;

  private double fuelLevel;

  public Car(double fuelLevel) {
    this.fuelLevel = fuelLevel;
  }

  @Override
  public void move(double dist) throws CannotMoveException {
    double spend = dist / Car.FUEL_EFFICIENCY;
    if (this.fuelLevel < spend) {
      throw new CannotMoveException();
    } else {
      this.fuelLevel -= spend;
    }
  }

  public void refuel(double amt) {
    this.fuelLevel += amt;
  }

  @Override
  public String toString() {
    return String.format("Car fuelLevel: %.2f", this.fuelLevel);
  }
}
```

Car.java

```java
class Bike extends Vehicle implements Foldable {
  private double totalDistance;
  private boolean folded;

  public Bike() {
    this.totalDistance = 0;
    this.folded = false;
  }

  @Override
  public void fold() {
    this.folded = true;
  }

  @Override
  public void unfold() {
    this.folded = false;
  }

  @Override
  public void move(double dist) throws CannotMoveException {
    if (this.folded) {
      throw new CannotMoveException();
    } else {
      this.totalDistance += dist;
    }
  }

  @Override
  public String toString() {
    return String.format("Bike distance: %.2f folded: %b", this.totalDistance, this.folded);
  }
}
```

Bike.java

```java
/**
 * This class encapsulates the RoadTrip object, that moves the vehicle a
 * specified distance.
 *
 * @author A0308798H
 */
class RoadTrip {
  private Vehicle vehicle;
  private double distance;

  public RoadTrip(Vehicle vehicle, double distance) {
    this.vehicle = vehicle;
    this.distance = distance;
  }

  public void complete() {
    try {
      this.vehicle.move(distance);
    } catch (CannotMoveException e) {
      System.out.println("This trip cannot be completed.");
    }
  }
}
```

NORMAL RoadTrip.java

cs2040s — ssh donko@pe111.comp.nus.edu.sg — 255x72

MessageDeliveryException.java | Messenger.java+ | TextMessenger.java | SecureMessenger.java | MessagingApp.java

```java
class MessageDeliveryException extends Exception {
  public MessageDeliveryException(String msg) {
    super(msg);
  }
}
```

MessageDeliveryException.java

```java
interface Messenger {
  void send(String msg, String recipient) throws MessageDeliveryException;

  String recv();
}
```

NORMAL Messenger.java[+]

```java
class TextMessenger implements Messenger {
  public TextMessenger() {
  }

  @Override
  public void send(String msg, String recipient) throws MessageDeliveryException {
    if (recipient.equals("")) {
      throw new MessageDeliveryException("Empty recipient");
    } else {
      System.out.printf("Sending message to %s: %s\n", recipient, msg);
    }
  }

  @Override
  public String recv() {
    return "Dummy message";
  }
}
```

TextMessenger.java

```java
class SecureMessenger implements Messenger {
  private Messenger messenger;

  public SecureMessenger(Messenger messenger) {
    this.messenger = messenger;
  }

  @Override
  public void send(String msg, String recipient) throws MessageDeliveryException {
    String encryptedMsg = String.format("Encrypted: %s", msg);
    this.messenger.send(encryptedMsg, recipient);
  }

  @Override
  public String recv() {
    return String.format("Decrypted: %s", this.messenger.recv());
  }
}
```

SecureMessenger.java

```java
class MessagingApp implements Messenger {
  private boolean encrypted;
  private TextMessenger textMsgr;
  private SecureMessenger secMsgr;

  public MessagingApp() {
    this.encrypted = false;

    this.textMsgr = new TextMessenger();
    this.secMsgr = new SecureMessenger(textMsgr);
  }

  public void setSecure(boolean encrypted) {
    this.encrypted = encrypted;
  }

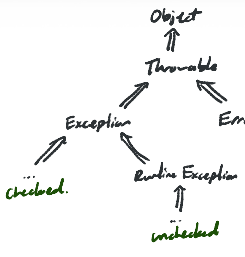
  @Override
  public void send(String msg, String recipient) {
    try {
      if (this.encrypted) {
        this.secMsgr.send(msg, recipient);
      } else {
        this.textMsgr.send(msg, recipient);
      }
    } catch (MessageDeliveryException e) {
      System.out.println("Unable to deliver message");
    }

  }

  public String recv() {
    if (this.encrypted) {
      return this.secMsgr.recv();
    } else {
      return this.textMsgr.recv();
    }
  }
}
```

MessagingApp.java

Format Specifier	Conversion Applied
%%	Inserts a % sign
%x %X	Integer hexadecimal
%t %T	Time and Date
%s %S	String
%n	Inserts a newline character
%o	Octal integer
%f	Decimal floating-point
%e %E	Scientific notation
%g	Causes Formatter to use either %f or %e, whichever is shorter
%h %H	Hash code of the argument
%d	Decimal integer
%c	Character
%b %B	Boolean
%a %A	Floating-point hexadecimal



Class can extend ≤1 class.
Class can implement ≥0 interfaces
Interface can extend ≥0 interfaces

public final class
⇒ Cannot be inherited from

public final method
⇒ Cannot be overridden